

## Problema de Asignación de Tareas mediante Ramificación y Poda

Este documento presenta la formalización matemática general para resolver el problema de asignar de forma óptima  $n$  personas a  $n$  tareas utilizando el esquema de Ramificación y Poda (*Branch and Bound*) de mínimo coste.

### 1. Transformación del Problema (De Maximizar a Minimizar)

El problema original busca encontrar una asignación que maximice la suma de los beneficios indicados en la matriz  $B[i, j]$  (donde  $B[i, j]$  es el beneficio de asignar la persona  $i$  a la tarea  $j$ ).

Para poder aplicar el esquema estándar de Ramificación y Poda de mínimo coste, transformamos la matriz de beneficios  $B$  en una matriz de costes  $C$  de la siguiente manera:

1. Buscamos el beneficio máximo absoluto de toda la matriz:

$$M = \max_{1 \leq i, j \leq n} B[i, j]$$

2. Definimos el coste de cada celda  $C[i, j]$  como la diferencia entre ese valor máximo  $M$  y el beneficio real. Esto representa la "oportunidad perdida" o coste de no elegir la opción óptima absoluta:

$$C[i, j] = M - B[i, j] \quad \forall i, j \in \{1, \dots, n\}$$

Dado que  $M \geq B[i, j]$ , todos los costes serán no negativos ( $C[i, j] \geq 0$ ). **Maximizar el beneficio total es matemáticamente equivalente a minimizar el coste total** en la nueva matriz  $C$ .

### a) Representación de Soluciones y Árbol de Búsqueda

#### 1. Representación de las Soluciones

Cualquier solución (parcial o completa) se representa mediante una tupla de decisiones de tamaño  $n$ :

$$X = (x_1, x_2, \dots, x_n)$$

Donde cada  $x_i \in \{1, 2, \dots, n\}$  indica el identificador de la tarea asignada a la persona  $i$ . Como la asignación debe ser unívoca (no se pueden repetir tareas), el vector final  $X$  debe ser una **permutación** del conjunto de tareas, es decir,  $x_i \neq x_k$  para todo  $i \neq k$ .

#### 2. Descripción del Árbol de Espacio de Soluciones

El espacio de búsqueda se estructura en un árbol de decisión general de la siguiente manera:

- **Raíz (Nivel 0):** Ninguna persona tiene asignada una tarea todavía.
- **Nivel  $i$  ( $1 \leq i \leq n$ ):** Representa la toma de decisión para la persona  $i$ .
- **Ramificación:** Un nodo en el nivel  $i - 1$  (donde ya se han asignado tareas a las personas  $1, \dots, i - 1$ ) se ramifica generando un nodo hijo por cada una de las tareas que todavía se encuentran libres (no asignadas en los niveles anteriores).
- **Hojas (Nivel  $n$ ):** Representan asignaciones completas y válidas de las  $n$  personas a las  $n$  tareas sin solapamientos.

## b) Función de Coste Real $c(x)$ a Minimizar

Para cualquier nodo  $x$  en el árbol de búsqueda (donde ya se han tomado decisiones fijas de asignación para las primeras  $k$  personas):

1. **Coste Acumulado Real  $g(x)$  (Lo que ya se ha pagado):** Es la suma de los costes reales de las decisiones ya tomadas e inamovibles desde la raíz hasta el nodo actual:

$$g(x) = \sum_{i=1}^k C[i, x_i]$$

2. **Función de Coste Real de una Solución Completa  $c(x)$ :** Cuando se alcanza una hoja (nivel  $n$ ), el coste real final de esa asignación completa y válida es:

$$c(x) = \sum_{i=1}^n C[i, x_i]$$

## c) Función de Coste Estimado $\hat{c}(x)$ (Cota Inferior)

La función de coste estimado  $\hat{c}(x)$  calcula una estimación optimista (el suelo mínimo absoluto) del coste de completar la asignación a partir de un nodo intermedio  $x$  (nivel  $k$ ).

Para que el algoritmo sea admisible (garantice encontrar el óptimo),  $\hat{c}(x)$  debe ser una **cota inferior válida (subestimación segura)** del coste real de cualquier hoja que descienda de él.

### "La Trampa Optimista"

Para calcular esta cota de forma muy rápida, liberamos temporalmente la restricción de que dos personas no puedan hacer la misma tarea.

Sea  $T_{\text{libres}} = \{1, \dots, n\} \setminus \{x_1, \dots, x_k\}$  el conjunto de tareas que aún no han sido asignadas en los niveles anteriores.

Para cada una de las personas restantes  $i \in \{k + 1, \dots, n\}$ , buscamos de manera independiente la tarea más barata disponible en  $T_{\text{libres}}$ , **ignorando por completo si varias personas coinciden en elegir la misma tarea:**

$$\text{MinCoste}(i) = \min_{j \in T_{\text{libres}}} C[i, j]$$

La cota inferior  $\hat{c}(x)$  se define como la suma del coste que ya hemos pagado más estas estimaciones optimistas e individuales:

$$\hat{c}(x) = g(x) + \sum_{i=k+1}^n \min_{j \in T_{\text{libres}}} C[i, j]$$

**Justificación de admisibilidad:** Como permitimos de manera ficticia que las personas restantes elijan su mejor opción sin preocuparse por los conflictos de asignación, este coste siempre será menor o igual al coste real de cualquier asignación legal (biyección) que descienda de este nodo.

#### d) Función de Poda $U(x)$ (Cota Superior)

La función de cota superior  $U(x)$  representa el coste real de una **solución factible completa y válida** (sin tareas repetidas) construida rápidamente mediante una heurística voraz a partir de las decisiones ya fijadas en el nodo actual  $x$ .

**Algoritmo Voraz para calcular  $U(x)$ :**

1. Inicializamos un conjunto temporal de tareas disponibles  $T_{\text{temp}} \leftarrow T_{\text{libres}}$  y un acumulador de coste adicional  $C_{\text{voraz}} \leftarrow 0$ .
2. Para cada persona restante  $i$  desde  $k + 1$  hasta  $n$ :
  - Le asignamos la tarea libre de  $T_{\text{temp}}$  que le suponga el menor coste posible:

$$j^* = \arg \min_{j \in T_{\text{temp}}} C[i, j]$$

- Añadimos ese coste al acumulador:  $C_{\text{voraz}} \leftarrow C_{\text{voraz}} + C[i, j^*]$ .
  - Eliminamos la tarea  $j^*$  de  $T_{\text{temp}}$  para asegurar que ninguna persona posterior pueda volver a elegirla (garantizando la validez legal de la asignación).
3. La cota superior se calcula como:

$$U(x) = g(x) + C_{\text{voraz}}$$